

System Design Fundamentals — Beginner Track

Target: Zero-to-solid foundation. No prior distributed systems experience required.

Goal: Understand the *why* before the *how*. Every concept is anchored to a real-world product you already use.

FDE Signal: Forward Deployed Engineers at Anthropic/OpenAI must explain complex systems to enterprise clients *and* build them. This track builds the vocabulary.

How to Use This Guide

Each topic follows this structure:

1. **What it is** — plain English definition
 2. **Why it matters** — the problem it solves
 3. **How it works** — mechanics with a diagram
 4. **Real-world example** — Netflix, YouTube, Stripe, etc.
 5. **Anti-patterns** — what goes wrong
 6. **Interview note** — what Anthropic/OpenAI interviewers actually test
-

PART 1 — Scaling & Architecture Basics

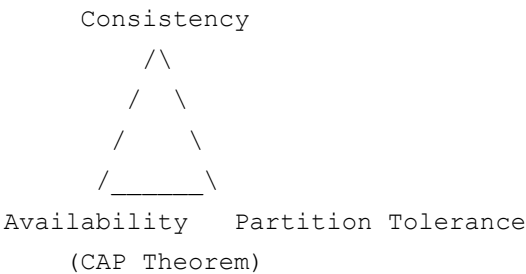
1. What Is System Design?

System design is the process of defining the architecture, components, and data flow of a system to meet functional and non-functional requirements.

Functional requirements = what the system *does* (e.g., “store a video”)

Non-functional requirements = how well it does it (latency, throughput, availability, consistency)

The Fundamental Trade-off Triangle



You can only have 2 of 3 in a distributed system. This is the first concept every interviewer expects you to know cold.

2. Vertical vs Horizontal Scaling

Vertical Scaling (Scale Up)

Add more resources (CPU, RAM) to the **same machine**.

Before: After:
[Server 2GB] → [Server 64GB]

Pros: Simple. No code changes.

Cons: Hardware limits. Single point of failure. Expensive.

Horizontal Scaling (Scale Out)

Add **more machines** instead of bigger ones.

```
Before:           After:
[Server]         → [Server] [Server] [Server]
                   ↑
                   Load Balancer
```

Pros: Near-infinite scale. Fault tolerant.

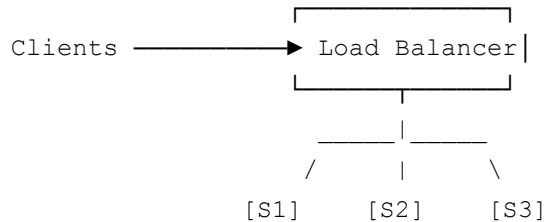
Cons: Complexity. Need load balancing. Stateless architecture required.

Real Example: Netflix started on vertical scaling (big servers). As they grew to 200M+ users, they moved to AWS with thousands of horizontal microservices.

Anti-pattern: Vertical scaling past the point where a single machine costs more than 10 horizontal machines. This is "elephant architecture" — expensive and fragile.

3. Load Balancing

A **Load Balancer** sits in front of your servers and distributes incoming requests.



Algorithms:

Algorithm	How it works	Use when
Round Robin	Rotate through servers equally	Uniform request sizes
Least Connections	Send to server with fewest active connections	Long-lived connections (WebSockets)
IP Hash	Hash client IP → same server	Session affinity needed
Weighted Round Robin	Servers get weight proportional to capacity	Mixed hardware

Real Example — YouTube

YouTube's load balancers route 500 hours of video uploaded per minute. They use geographic load balancing (send to nearest data center) then internal load balancing within the data center.

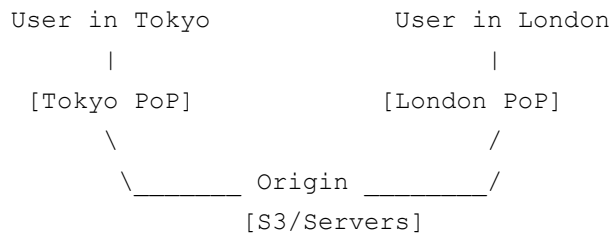
Anti-patterns:

- **No health checks:** Load balancer sends traffic to dead servers
- **Session stickiness without fallback:** One server dies, user loses session
- **Not load-balancing the load balancer itself:** The LB itself becomes a single point of failure (use DNS round-robin or Anycast)

Interview Note (FDE): Be ready to discuss L4 vs L7 load balancing. L4 operates at TCP level (faster, less smart). L7 operates at HTTP level (can route by URL path, headers — smarter but slower).

4. CDN (Content Delivery Network)

A CDN is a geographically distributed network of servers (called **edge nodes** or **PoPs**) that cache and serve content close to users.



How it works:

1. User requests `https://cdn.netflix.com/thumbnail.jpg`
2. DNS resolves to nearest CDN edge node
3. Edge checks cache. **Cache hit** → serve immediately. **Cache miss** → fetch from origin, cache, serve.

What CDNs cache:

- Static assets: images, JS, CSS, fonts
- Video segments (Netflix, YouTube)
- API responses (with short TTLs)
- Entire HTML pages (Cloudflare edge workers)

Real Example — Netflix Open Connect

Netflix built its own CDN called **Open Connect**. They place physical appliances *inside* ISP data centers. When you hit play, you're often streaming from a box in your own ISP's building — not from Netflix's origin. This handles ~15% of all North American internet traffic.

Anti-patterns:

- **Caching dynamic data without TTL:** Users see stale prices, stock info
- **No cache invalidation strategy:** Deploying a bug fix but old JS is cached everywhere
- **Not using CDN for APIs:** API responses can be cached too (with short TTLs)

Interview Note: Know the difference between **push CDN** (you upload content proactively) vs **pull CDN** (CDN fetches on first request). Understand `Cache-Control: max-age`, `ETag`, and `Vary` headers.

5. Caching

Caching stores frequently-accessed data in fast storage (memory) to avoid repeated expensive operations (DB queries, API calls, computations).

```
Request → [Cache] → Hit? → Return data
                        ↓
                      Miss
                        ↓
[Database] → Store in cache → Return data
```

Cache Patterns:

Cache-Aside (Lazy Loading) — Most Common

```
App reads from cache → miss → read DB → write to cache → return
App writes to DB → invalidate/update cache
```

Write-Through

```
App writes → write to cache AND DB simultaneously
Reads always hit cache (always warm)
```

Write-Behind (Write-Back)

```
App writes → write to cache → async write to DB
Fast writes. Risk: data loss if cache crashes before DB write
```

Read-Through

```
Cache sits in front of DB. App only talks to cache.
Cache handles DB reads transparently.
```

Real Example — Twitter Timeline

Twitter uses Redis to cache the timelines of active users. When you follow someone

famous, their tweet is **fan-out** written to millions of followers' cached timelines instantly. For inactive users, timeline is reconstructed lazily from DB.

Cache Eviction Policies:

Policy	Behavior	Best for
LRU (Least Recently Used)	Evict longest-unused item	General use
LFU (Least Frequently Used)	Evict least-accessed item	Hot content
FIFO	Evict oldest-added item	Simple queues
TTL	Evict after time expires	Session data

Anti-patterns:

- **Cache stampede (Thundering Herd):** Cache expires → thousands of requests hit DB simultaneously. Fix: Probabilistic Early Expiration or mutex locks.
 - **Stale data:** Caching without invalidation strategy
 - **Caching everything:** Memory is finite. Cache selectively.
 - **No eviction policy:** Cache grows until OOM crash
-

6. Databases — SQL vs NoSQL

SQL (Relational)

Data in tables with rows and columns. Schema is fixed. Relations via foreign keys.

```
Users:           Orders:
id | name          id | user_id | amount
1  | Alice         1  | 1       | $50
2  | Bob           2  | 1       | $30
```

ACID guarantees: Atomicity, Consistency, Isolation, Durability

Examples: PostgreSQL, MySQL, SQLite, Aurora

Use when: Financial transactions, user accounts, anything with complex joins, compliance requirements.

NoSQL

Multiple types:

Type	Example	Best for
Document	MongoDB, Firestore	User profiles, product catalogs
Key-Value	Redis, DynamoDB	Sessions, caching, leaderboards
Wide-Column	Cassandra, HBase	Time-series, event logs, IoT
Graph	Neo4j, Neptune	Social networks, recommendations

BASE guarantees: Basically Available, Soft state, Eventually consistent

Real Example:

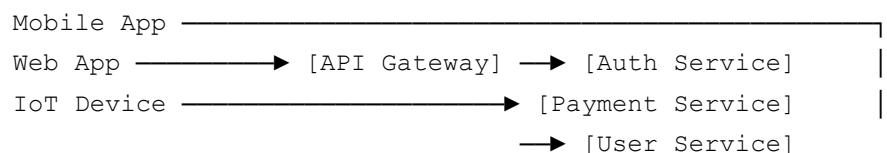
- **Uber** uses a mix: PostgreSQL for billing (ACID), Cassandra for trip location data (massive writes), Redis for real-time driver locations
- **Instagram** started on PostgreSQL, added Cassandra for feed storage as they scaled

Anti-patterns:

- **Using NoSQL just because it's "modern":** SQL is often faster and simpler
- **No indexing:** Full table scans kill performance at scale
- **Storing BLOBs in DB:** Store files in S3, store the URL in DB

7. API Gateway

An API Gateway is the **single entry point** for all client requests to your backend services.



What an API Gateway does:

- **Authentication & Authorization** — Validate JWT tokens

- **Rate Limiting** — Limit requests per user/IP
- **SSL Termination** — Handle HTTPS, forward HTTP internally
- **Request Routing** — Route `/payments` to payment service
- **Load Balancing** — Distribute across service instances
- **Request/Response Transformation** — Protocol translation (REST to gRPC)
- **Logging & Monitoring** — Centralized observability
- **Caching** — Cache common responses

Real Examples: AWS API Gateway, Kong, Nginx, Envoy, Apigee, Cloudflare Gateway

Interview Note: The API Gateway pattern is central to microservices. Differentiate between **API Gateway** (external-facing) and **Service Mesh** (internal service-to-service). An API Gateway is like your building's front door. A Service Mesh is the internal communication network between rooms.

8. Rate Limiting

Rate limiting controls how many requests a user or IP can make in a given time window.

Why: Prevent abuse, DDoS protection, ensure fair usage, control infrastructure cost.

Algorithms:

Token Bucket — Most Common

```
Bucket has N tokens. Each request uses 1 token.
Tokens refill at rate R per second.
Burst allowed (can use saved tokens).
```

Fixed Window Counter

```
Count requests per minute/hour.
Reset counter at window boundary.
Problem: "boundary burst" — 100 req at 11:59, 100 at 12:00 = 200 in 2 seconds.
```

Sliding Window Log

```
Store timestamp of each request.
```


Count requests in last N seconds.
Most accurate. High memory use.

Sliding Window Counter

Hybrid: combine current and previous window with weighted formula.
Good accuracy, lower memory.

Real Example — Stripe API Stripe rate limits by API key: 100 reads/second, 100 writes/second. Exceeding returns HTTP 429. They use the Token Bucket algorithm stored in Redis.

Where to implement:

- API Gateway layer (most common)
- Application code
- Reverse proxy (Nginx)

Anti-patterns:

- Rate limiting only by IP (easy to bypass with proxies; use user ID + IP)
- No rate limit headers in response (clients can't implement backoff)
- Forgetting to rate limit internal services too

9. Queueing (Message Queues)

A message queue decouples producers (senders) from consumers (receivers) with an async buffer.

Producer → [Queue] → Consumer
(Web Server) (Worker Service)

Why: Handle spikes. Don't drop work. Enable async processing.

Real Example — Email on Sign-up

Without queue (synchronous):
User signs up → Server sends email → User waits 3 seconds → Response

With queue (asynchronous):

User signs up → Server puts "send email" in queue → Response (fast!)

Worker picks up message → Sends email

Popular Message Queues:

System	Best for
RabbitMQ	Complex routing, low latency
Apache Kafka	High-throughput event streaming
AWS SQS	Managed, simple queues
Redis (Streams)	Lightweight queuing on existing Redis
Google Pub/Sub	Managed pub/sub at scale

Key Concepts:

- **Producer:** Sends messages
- **Consumer:** Receives and processes messages
- **Queue:** FIFO buffer (point-to-point)
- **Topic/Exchange:** Pub/sub broadcast
- **Dead Letter Queue (DLQ):** Where failed messages go after N retries

Anti-patterns:

- **No DLQ:** Failed messages are silently dropped
- **Queue as database:** Don't store long-term data in queues
- **Not idempotent consumers:** If consumer processes same message twice (at-least-once delivery), it must be safe

10. Monolith vs Microservices

Monolith

All code in one deployable unit.

```
[Single Application]
- User Module
- Payment Module
- Notification Module
- Analytics Module
```

Pros: Simple to develop, test, deploy early on. No network overhead. ACID transactions easy.

Cons: Scales as a unit (can't scale only payment). One bug can crash everything. Team size doesn't scale.

Microservices

Each module is a separate, independently deployable service.

```
[User Service] [Payment Service] [Notification Service] [Analytics Service]
      ↓              ↓              ↓              ↓
      [DB]          [DB]          [Queue]          [DB]
```

Pros: Independent scaling. Independent deployment. Technology flexibility. Team autonomy.

Cons: Network complexity. Distributed transactions hard. More infrastructure overhead. Harder to debug.

Real Example — Amazon

Amazon famously migrated from a monolith to microservices in the early 2000s. Jeff Bezos mandated that every team expose their data/functionality via service APIs. This became the foundation for AWS (they dogfooded their own infrastructure services).

When to use what:

- **Startup/MVP:** Monolith. Ship fast.
- **Scaling team/product:** Start extracting high-load services (Strangler Fig pattern)
- **Large org with multiple teams:** Microservices per team

Anti-patterns:

- **Distributed monolith:** Microservices that are so tightly coupled they must deploy together
- **Micro-microservices:** Function-per-service causes explosion of network calls
- **Ignoring data ownership:** Microservices sharing a database is a monolith with

11. CAP Theorem

In any distributed data store, you can guarantee at most **2 of 3**:

- **Consistency** — Every read sees the most recent write
- **Availability** — Every request gets a (non-error) response
- **Partition Tolerance** — System works even if network fails between nodes

Network partitions always happen. So you really choose between CP and AP.

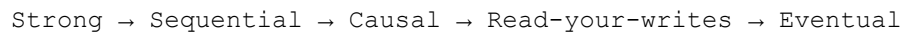
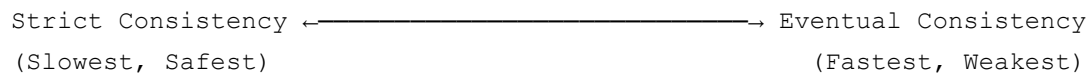
Choice	Behavior	Examples
CP	Reject requests when uncertain	HBase, ZooKeeper, MongoDB (primary)
AP	Return possibly stale data	DynamoDB, Cassandra, CouchDB

Real Example:

- **Bank balance check (CP):** Better to show error than wrong balance
- **Netflix recommendation (AP):** Showing slightly stale “Continue Watching” is fine
- **Shopping cart (AP → eventual):** Amazon chose availability over consistency for shopping carts (Dynamo paper, 2007)

Interview Note: Interviewers expect you to *justify* your DB choice using CAP. “I’d use Cassandra for its AP model because this feature prioritizes availability over strict consistency” shows FDE-level thinking.

12. Consistency Models (Simplified)



Model	Guarantee	Example

Strong	All nodes see same data immediately	Single-node DB
Sequential	All nodes see operations in same order	Paxos-based systems
Causal	Causally related operations ordered	Git (branches)
Read-your-writes	You always see your own writes	User profile updates
Eventual	All nodes converge eventually	DNS, Cassandra

Eventual Consistency real example: When you update your Twitter bio, it might take a few seconds to appear worldwide. That's eventual consistency. Acceptable for profile data. Not acceptable for bank balance.

13. Replication

Replication = copying data to multiple servers for availability and performance.

```

Primary (Write) — replicates —> Replica 1 (Read)
                                ▶ Replica 2 (Read)
                                ▶ Replica 3 (Read) [different region]

```

Types:

- **Synchronous:** Primary waits for replica acknowledgment. Strong consistency. Slower writes.
- **Asynchronous:** Primary doesn't wait. Faster writes. Risk of data loss if primary crashes.
- **Semi-synchronous:** Wait for at least one replica. Balance.

Real Example — MySQL at Facebook Facebook ran MySQL with one primary (writes) and many read replicas (reads). 90%+ of their traffic was reads, so reads scaled horizontally while writes stayed on primary. They had 9:1 read-to-write ratio.

14. Fault Tolerance

A fault-tolerant system continues operating correctly even when components fail.

Techniques:

- **Redundancy:** Multiple instances of every component
- **Failover:** Automatically switch to backup on failure
- **Health checks:** Continuously verify component health
- **Circuit breaker:** Stop calling a failing service
- **Graceful degradation:** Return partial results instead of error

Real Example — Netflix Chaos Engineering Netflix intentionally kills random servers in production (Chaos Monkey) to ensure the system is truly fault tolerant. If it can survive deliberate failures, it survives real ones.

Anti-patterns:

- **Single Point of Failure (SPOF):** Any component with no redundancy
- **Cascading failures:** One service fails, overwhelms others → total outage

PART 2 — Databases & Storage Basics

15. ACID Properties

Every database transaction should satisfy:

Property	Meaning	Example
Atomicity	All-or-nothing. Partial writes don't happen.	Transfer \$100: debit + credit both succeed or both fail
Consistency	DB moves from one valid state to another.	Balance never goes negative
Isolation	Concurrent transactions don't interfere.	Two users booking last seat don't both get it
Durability	Committed data survives crashes.	Power cut after commit = data still there

16. Indexing

An index is a data structure that speeds up data retrieval.

Without index: `SELECT * WHERE email = 'alice@x.com'`

→ Scan all 10M rows (full table scan) → Slow

With index on email column:

→ B+ Tree lookup → $O(\log n)$ → Fast

Types:

- **B+ Tree Index:** Default. Range queries, ORDER BY, LIKE 'prefix%'
- **Hash Index:** Exact match only. Fastest for equality. No ranges.
- **Full-Text Index:** Text search. Used in Elasticsearch, PostgreSQL tsvector.
- **Composite Index:** Multiple columns. Order matters. `(user_id, created_at)` helps
`WHERE user_id=X ORDER BY created_at.`

Anti-patterns:

- **Too many indexes:** Speeds up reads but slows writes (every insert updates all indexes)
 - **Wrong index column order:** `(city, state)` index won't help `WHERE state = 'CA'`
 - **Not using EXPLAIN:** Verify your query actually uses the index
-

17. SQL vs NoSQL — When to Use Which

Use SQL when:

- Data has clear relationships (users → orders → products)
- ACID transactions required (banking, payments)
- Schema is well-defined and won't change much
- Complex queries with JOINS
- Team is small and SQL expertise available

Use NoSQL when:

- Need to scale horizontally (billions of records)
- Schema is flexible / changes often
- Data is document-like (user profile with nested objects)

- Need very high write throughput
 - Specific access patterns (always access by user_id)
-

18. Object Storage (S3 Basics)

Object storage stores files as immutable objects with a key (path) and metadata.

```
Bucket: my-videos
├─ user-123/video-456.mp4    ← key
├─ user-123/thumbnail.jpg
└─ user-789/video-101.mp4
```

Properties:

- No file system hierarchy (flat, but keys can simulate paths)
- Immutable objects (can't edit in place, must upload new version)
- Massive scale (exabytes)
- HTTP-based access (presigned URLs)
- Cheap (\$0.023/GB/month on S3)

Real Example:

- Every photo on Instagram is stored in S3 (or Meta's equivalent, Haystack)
- Netflix stores master video files in S3, transcodes to multiple resolutions
- Airbnb stores all listing photos in S3

Anti-patterns:

- Storing S3 objects with unpredictable access patterns (use prefix for even distribution)
 - Serving large files directly from S3 without CDN (latency + cost)
 - Storing sensitive data without encryption at rest
-

19. Data Modeling Basics

Normalization — Eliminate redundancy. More tables, more joins.


```
-- Normalized
Users(id, name, email)
Orders(id, user_id, product_id, amount)
Products(id, name, price)
```

Denormalization — Embed redundant data for read speed. Fewer joins.

```
// Denormalized (MongoDB document)
{
  "order_id": "123",
  "user_name": "Alice",      ← duplicated from User
  "product_name": "Laptop", ← duplicated from Product
  "amount": 999
}
```

Rule of thumb: Normalize for writes (OLTP). Denormalize for reads (analytics/OLAP).

PART 3 — Communication & APIs Basics

20. REST vs gRPC

REST

- Uses HTTP/1.1 or HTTP/2
- Resources are URLs: `GET /users/123`
- Data format: JSON (text-based, human-readable)
- Stateless
- Universal: works with browsers, curl, any language

gRPC

- Uses HTTP/2
- Defines services with Protocol Buffers (binary format)
- Strongly typed contracts
- 2-10x faster than REST (binary serialization)

- **Bi-directional streaming**

```
// gRPC service definition
service UserService {
  rpc GetUser (UserRequest) returns (UserResponse);
  rpc StreamUpdates (UserRequest) returns (stream Update);
}
```

When to use:

- **REST:** External APIs, browser clients, public APIs, simplicity
- **gRPC:** Internal microservice communication, high performance, streaming

Real Example: Google uses gRPC for all internal service communication. Kubernetes API server uses gRPC internally, REST externally.

21. WebSockets vs HTTP

HTTP (Request-Response):

```
Client — GET /messages —> Server
Client <— 200 [messages] — Server
(connection closed)
```

WebSocket (Full-Duplex):

```
Client <— WS Upgrade — Server
      <— bidirectional messages —>
(connection stays open)
```

Use WebSockets for: Chat, live dashboards, collaborative editing, gaming, real-time notifications

Use HTTP/SSE for: One-way push (stock tickers, news feeds), simpler implementation

Real Example:

- Slack uses WebSockets for real-time message delivery
 - Google Docs uses WebSockets (via Operational Transformation) for collaborative editing
-

22. JWT (JSON Web Tokens)

JWTs are self-contained tokens for authentication/authorization.

```
Header.Payload.Signature
```

```
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiIxMjMifQ.abc123
```

```
Header: {"alg": "HS256", "typ": "JWT"}
```

```
Payload: {"userId": "123", "role": "admin", "exp": 1234567890}
```

```
Signature: HMAC-SHA256(header + payload, secret)
```

Flow:

1. User logs in → Server creates JWT → Returns to client
2. Client stores JWT (localStorage or httpOnly cookie)
3. Client sends JWT in Authorization header: Bearer <token>
4. Server verifies signature → Extracts userId → Proceeds

Anti-patterns:

- Storing JWT in localStorage (XSS vulnerable). Use httpOnly cookies.
 - No expiration (exp claim). Stolen tokens last forever.
 - Putting sensitive data in payload (base64 is not encryption)
 - Not verifying on every request
-

23. OAuth 2.0

OAuth is an authorization framework that lets users grant apps access to their data without sharing passwords.

1. User clicks "Login with Google"
2. App redirects to Google OAuth server
3. User approves permissions
4. Google returns authorization code
5. App exchanges code for access_token + refresh_token
6. App uses access_token to call Google APIs

Roles:

- **Resource Owner:** User
- **Client:** Your app
- **Authorization Server:** Google/GitHub (issues tokens)
- **Resource Server:** Google API (protects data)

Real Example: Every "Login with Google/GitHub/Apple" button uses OAuth 2.0.

24. CORS (Cross-Origin Resource Sharing)

CORS is a browser security mechanism that controls which origins can make requests to your API.

Frontend: `https://myapp.com`

API: `https://api.myapp.com`

Browser blocks request: different origin!

Unless API responds with: `Access-Control-Allow-Origin: https://myapp.com`

Preflight Request: For non-simple requests, browser sends `OPTIONS` request first to check if CORS is allowed.

Anti-patterns:

- `Access-Control-Allow-Origin: *` on authenticated endpoints (allows any site to make requests)
 - Forgetting to allow `Authorization` header in `Access-Control-Allow-Headers`
-

25. Idempotency

An operation is idempotent if performing it multiple times produces the same result as performing it once.

Idempotent: `DELETE /users/123` (delete again → still deleted, same state)

Idempotent: `PUT /users/123 {name: "Alice"}` (update again → same result)

Not Idempotent: `POST /orders` (create again → duplicate order!)

Why it matters: Networks are unreliable. Clients retry failed requests. Without idempotency, retries cause duplicate payments, double charges, etc.

Solution — Idempotency Keys:

```
POST /payments
Idempotency-Key: client-generated-uuid-123
{amount: 100}
```

Server: store key + result. If key seen again → return cached result.

Real Example: Stripe requires `Idempotency-Key` header on all payment creation requests. If your server crashes after creating a payment but before returning response, you safely retry with same key.

PART 4 — Reliability Basics

26. SLA / SLO / SLI

These define reliability targets and measurements:

Term	Meaning	Example
SLI (Indicator)	What you measure	99.95% of requests return <200ms
SLO (Objective)	Target you set internally	We aim for 99.9% availability
SLA (Agreement)	Contract with customer (with penalties)	We guarantee 99.9% or you get credit

The hierarchy: SLA > SLO > SLI

Error Budget:

```
SLO: 99.9% availability
Error budget: 0.1% = ~8.7 hours/year of allowed downtime
```

```
Deploy frequently within error budget.
Freeze deployments when budget is exhausted.
```

Real Example: AWS S3 SLA is 99.9% monthly uptime. If they breach it, you get service credits. Their internal SLO is much higher (99.99%+) as a safety buffer.

27. Health Checks

Health checks allow load balancers and orchestrators to know if a service is healthy.

Liveness Probe: Is the service running at all?

`GET /health → 200 OK`

Readiness Probe: Is the service ready to accept traffic? (DB connected, warmed up?)

`GET /ready → 200 OK`

Kubernetes uses both. Liveness failure → restart pod. Readiness failure → remove from load balancer pool.

Anti-pattern: Health check that just returns 200 without actually checking DB connection, cache availability, etc.

28. Logging, Metrics, Tracing (Observability Triangle)

`Observability = Logs + Metrics + Traces`

Pillar	What it tells you	Tools
Logs	What happened (events)	ELK Stack, Loki, CloudWatch
Metrics	How the system behaves (numbers over time)	Prometheus, Datadog, CloudWatch
Traces	How a request flows through services	Jaeger, Zipkin, OpenTelemetry

Real Example: When a Netflix user can't play a video:

- **Logs** show the specific error message and stack trace
- **Metrics** show latency spike on CDN edge at that time
- **Traces** show the request path: CDN → API Gateway → Content Service → DB → timeout

FDE Note: Anthropic and OpenAI care deeply about observability because LLM inference is expensive and unpredictable. You need to trace exactly which prompt caused which output and at what cost.

29. Circuit Breaker

A circuit breaker pattern prevents cascading failures by stopping calls to a failing service.

States:

CLOSED → (normal, requests pass through)

| error rate > threshold



OPEN → (all requests fail fast, don't call service)

| after timeout, try one request



HALF-OPEN → (test if service recovered)

| success → CLOSED

| failure → OPEN

Real Example: Netflix's Hystrix (now resilience4j) wraps every service call in a circuit breaker. If the recommendation service is down, the circuit opens and Netflix shows a generic "Popular Movies" list instead of timing out.

30. Autoscaling

Automatically adjust the number of instances based on load.

Horizontal Pod Autoscaler (Kubernetes):

When CPU > 70% → add pods

When CPU < 30% → remove pods

Min: 2 pods, Max: 20 pods

Scaling triggers:

- CPU utilization
- Memory utilization
- Custom metrics (queue depth, requests/second)
- Schedule-based (pre-scale before Super Bowl traffic spike)

Real Example: Netflix pre-scales for new show releases. When Stranger Things season drops, they scale up before, not in reaction to, the traffic spike.

Anti-patterns:

- **Scaling too slowly:** By the time new instances are ready, traffic has already caused failures
 - **No scale-in protection:** Terminating instances that still have active connections
 - **Scaling the wrong metric:** CPU might not correlate with actual load (I/O-bound services)
-

PART 5 — The FDE-Specific Layer

31. Why System Design Matters for FDE Roles

Forward Deployed Engineers are not pure backend engineers. You are the technical bridge between Anthropic/OpenAI and enterprise customers. This means:

1. **You design systems that use AI:** RAG pipelines, agent workflows, compliance automation
2. **You explain architecture to non-engineers:** CTO, CISO, compliance officers
3. **You understand customer constraints:** Data residency, existing infrastructure, budget
4. **You build proof-of-concepts and production systems**

The FDE System Design Framework (Remember: SCARE)

- **Scale** — How much load? Current and projected?
- **Consistency** — How fresh does data need to be?
- **Availability** — What's the cost of downtime?
- **Reliability** — How do we fail gracefully?
- **Evolvability** — Can we extend this without rewriting?

Common FDE Interview Prompt Types at Anthropic/OpenAI:

1. "Design a rate limiter for the Claude API" ← distributed systems + Redis
2. "Design a compliance document ingestion pipeline" ← queues + RAG + vector DB

3. "Design a multi-tenant LLM serving system" ← isolation + cost attribution
 4. "How would you build a safe content moderation pipeline?" ← safety-first architecture
-

Quick Reference: Beginner Cheat Sheet

Load Balancing → Distribute traffic across servers
CDN → Serve content from edge, close to users
Caching → Store frequently-used data in memory
Rate Limiting → Control request volume per user
API Gateway → Single entry point for all APIs
Message Queue → Async processing, decouple services
Monolith → Simple, one unit, hard to scale
Microservices → Complex, independently scalable
CAP Theorem → Pick 2: Consistency, Availability, Partition Tolerance
ACID → SQL guarantees (Atomic, Consistent, Isolated, Durable)
BASE → NoSQL guarantees (Basically Available, Soft state, Eventually consistent)
SLI/SLO/SLA → Measure → Target → Contract
Circuit Breaker → Stop calling failing services
Health Checks → Know if service is alive and ready
Idempotency → Safe to retry without side effects

Next: [Intermediate Track →](#) — Distributed systems depth, database internals, and real design walkthroughs.